Project - Phase 3

Name	Student Number
Diogo Martins	1221223
Francisco Osorio	1220846
Joao Pinto	1220663
Francisco Reis	1201373
Marco Marques	1250685

Introduction

In this report, its presentend the phase 3 of the DESOFS project, which consists in the development of the software, following the best practices and security measures defined in the previous phases. The main goal of this phase is to implement the software according to the requirements and design defined in the previous phases, while ensuring that the code is maintainable, scalable and secure.

Project Overview

TechStore consists of the development of a secure e-commerce platform designed to support online product sales, order management, and delivery operations. The system provides a set of RESTful services and a web-based user interface, enabling customers, managers, and carriers to interact with the platform according to their assigned roles.

Additionally, the system provides administrative functionalities that allow managers to maintain product catalogs, manage categories, monitor orders, and oversee platform operations.

Key Features

- User Management: Handling user registration, authentication, account verification, password recovery, and role-based access control for customers, managers, and carriers.
- Product Management: Allowing managers to create, update, and remove products and categories while providing customers with access to product information and availability.
- Shopping Cart Management: Enabling customers to manage cart contents before completing purchases.
- Order Management: Supporting order creation, tracking, and status management throughout the purchase lifecycle.
- Delivery Operations: Allowing carriers to view assigned deliveries, manage order pickups, and update delivery-related information.
- Administrative Functions: Providing managers with access to operational functionality, system monitoring capabilities, and business-related management features.

- Security Controls: Implementing secure authentication, authorization, HTTPS communication, restricted CORS policies, Content Security Policy (CSP), and additional browser security protections aligned with industry best practices.

The system architecture consists of a Next.js web frontend, a Spring Boot REST API backend, and a relational database for persistent data storage. An NGINX reverse proxy is used to route traffic and enforce secure communication. The application is designed to be scalable, maintainable, and secure while providing a reliable online shopping experience for all user roles.

Full Workflow

Overview

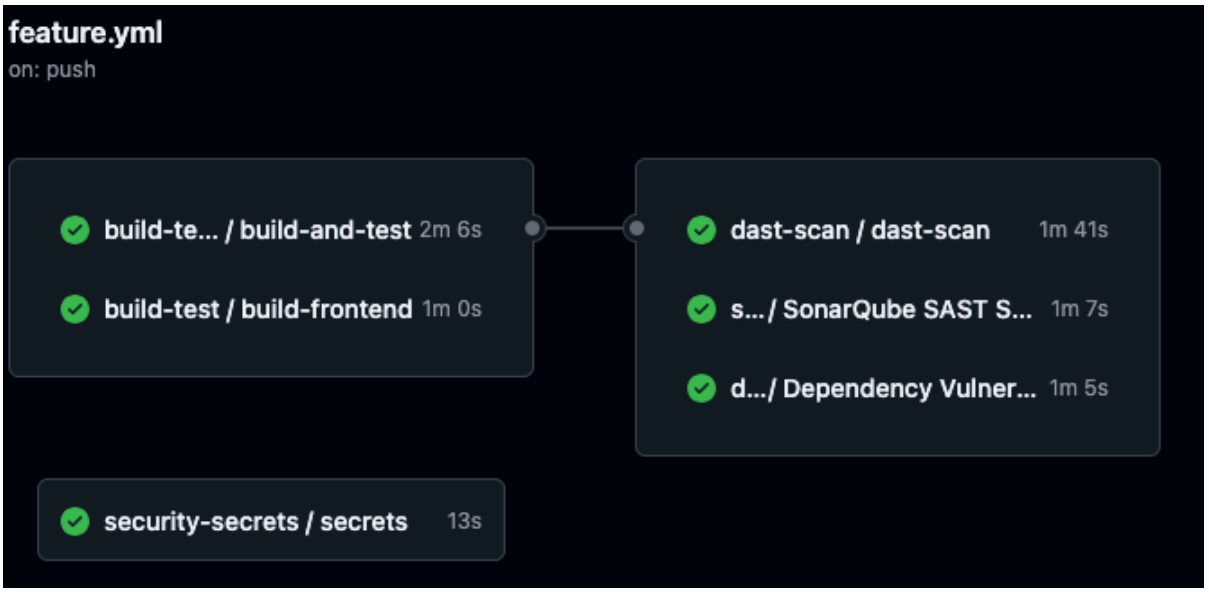
Compared to the previous report, the CI/CD workflows were extended to support the frontend application across all environments.

The following improvements were introduced:

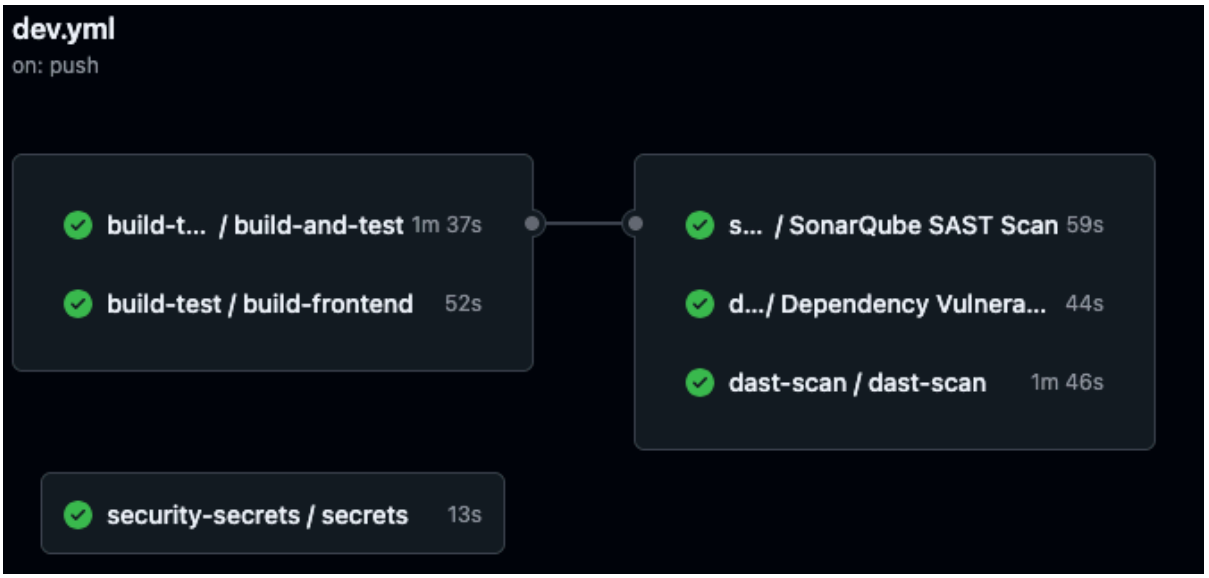
- **Frontend build integration**: Frontend build stages were added to the Feature, Dev, and Main workflows, ensuring that frontend changes are continuously validated alongside backend changes.
- **Full-stack deployment**: The Main workflow was extended to deploy the frontend application together with the backend, providing a complete automated deployment process.
- **Functional testing**: Functional tests were added to the Main workflow to validate end-to-end system behavior after the build and deployment stages.

These enhancements strengthen the CI/CD process by providing full-stack validation, automated frontend deployment, and additional quality assurance through functional testing before production releases.

Example of workflow run to feature branch



Example of workflow run to dev branch



Example of workflow run to main branch

Requirements Implemented

Functional Requirements

ID	Requirement	Status
FR1	Allow anonymous users to register as a customer	Done
FR2	Allow users to log in and log out	Done
FR3	Provide password recovery functionality	Done
FR4	Allow authenticated users to refresh JWT tokens before expiration	Done
FR5	Allow managers to invite new users (managers or carriers)	Done
FR6	Allow invited users to complete registration	Done
FR7	Display a list of available products	Done
FR8	Allow users to search products by name	Done
FR9	Display product details (price, description, stock)	Done
FR10	Allow customers to add products to the cart	Done
FR11	Allow customers to remove products from the cart	Done
FR12	Allow customers to update product quantities in the cart	Done

ID	Requirement	Status
FR13	Automatically calculate cart totals	Done
FR14	Allow customers to place orders	Done
FR15	Validate product stock before confirming orders	Done
FR16	Store user order history	Done
FR17	Allow customers to view order status	Done
FR18	Send order confirmation emails	Done
FR19	Allow carriers to view orders ready for pickup	Done
FR20	Display relevant order information for pickup	Done
FR21	Allow carriers to mark an order as picked up	Done
FR22	Allow managers to add new products	Done
FR23	Allow managers to edit product information	Not Done
FR24	Allow managers to manage product categories	Done
FR25	Allow managers to update product stock levels manually	Done
FR26	Allow managers to view and filter all customer orders	Done
FR27	Allow managers to create backups of products, categories, and orders	Partially Done

Non-Functional Requirements

ID	Requirement	Status
NFR1	API must be accessible only via HTTPS (TLS 1.2+) in non-local environments	Done
NFR2	Passwords must be hashed using a strong adaptive algorithm (e.g., BCrypt)	Done
NFR3	System must enforce RBAC with deny-by-default authorization	Done
NFR4	System must mitigate common web vulnerabilities (SQLi, XSS, CSRF where applicable)	Done
NFR5	Security-relevant actions must be logged with timestamp and user context	Done
NFR6	Two-factor authentication is mandatory for all users	Partially Done
NFR7	System must handle at least 100 concurrent requests with <500ms response time	Done
NFR8	Codebase must follow clean architecture principles	Done
NFR9	CI/CD must run automated build, tests, and security checks	Done
NFR10	Application must be containerized with non-root execution	Done

ID	Requirement	Status
NFR11	API must follow REST conventions with consistent error handling	Done
NFR12	OpenAPI documentation must be maintained for all endpoints	Done
NFR13	Dependency scanning must block critical vulnerabilities	Done
NFR14	Automated tests must ensure ≥80% coverage in core layers	Done
NFR15	Secrets must not be stored in source code; secret scanning enabled	Done

Security Requirements

ID	Requirement	Status
SR1	Multi-Factor Authentication (MFA) required for authentication	Done
SR2	Lock accounts after 5 failed login attempts with cooldown	Not Done
SR3	Passwords must be at least 12 characters long	Done
SR4	Send email confirmation after registration	Done
SR5	Role-Based Access Control (RBAC) must be enforced	Done
SR6	Sessions expire after inactivity	Done
SR7	Rate limiting must be implemented on entry endpoints	Done
SR8	Sensitive data must be encrypted in transit and at rest	Done
SR9	Passwords must be hashed with bcrypt or equivalent	Done
SR10	GDPR compliance for personal data handling	Not Done
SR11	Input validation must prevent SQL injection and XSS	Done
SR12	Validate all user-submitted data formats	Done
SR13	Validate transactional consistency (orders, stock, totals)	Done
SR14	Secure storage of sensitive logs	Done
SR15	Logs must be backed up in 3 locations (local + 2 cloud)	Done

Production Deployment

For the production environment, an Azure Virtual Machine was provisioned to host both the backend API and the frontend application.

To expose the applications to end users, Nginx was configured as a reverse proxy and web server, routing incoming requests to the appropriate services and serving the frontend application.

Additionally, HTTPS was enabled for both the frontend and backend endpoints using *Let's Encrypt* certificates, ensuring encrypted communication between clients and the deployed services. This configuration improves security by protecting data in transit and providing trusted SSL/TLS certificates.

The final production setup includes:

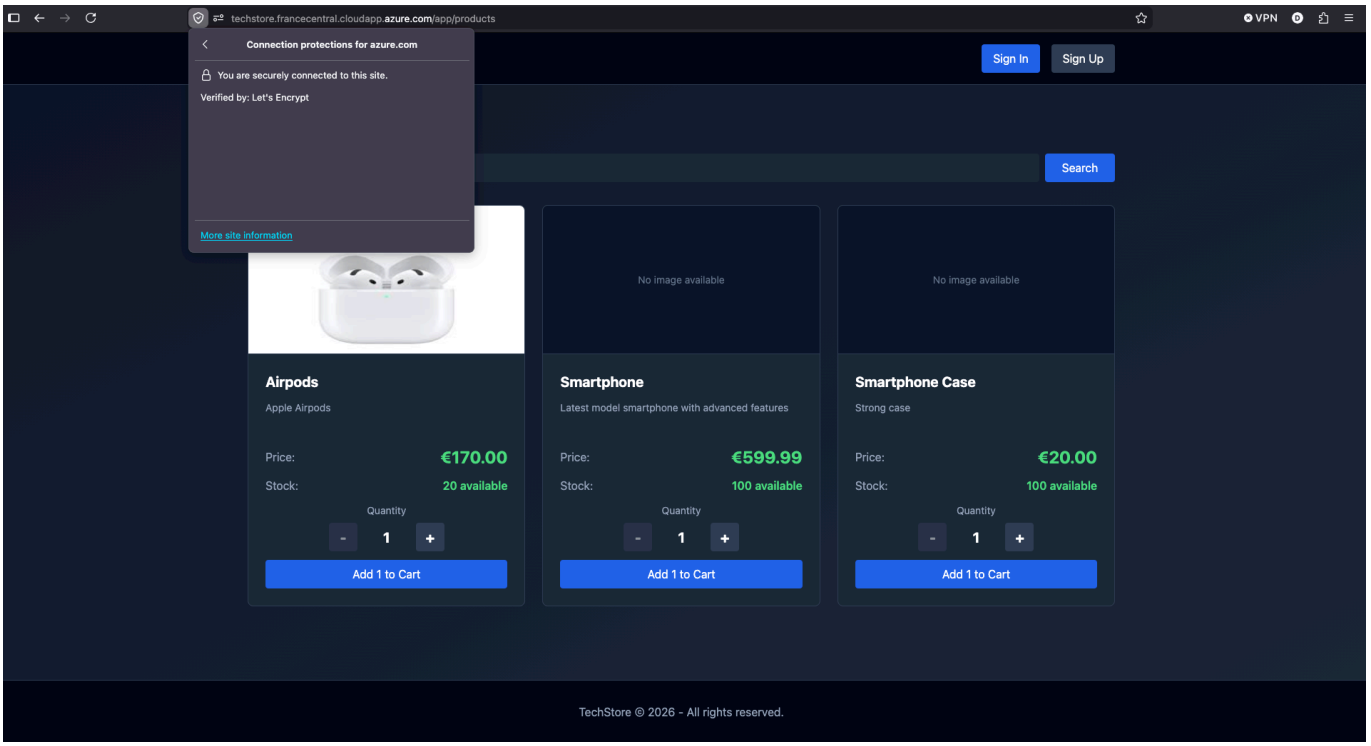
- Azure VM hosting the application infrastructure.
- Nginx configuration for application routing and reverse proxying.
- Frontend and backend applications deployed on the same environment.
- HTTPS enabled for both services.
- Automatic certificate management through *Let's Encrypt*.

Nginx Configuration

```
azureuser@desofs:~$ sudo cat /etc/nginx/sites-available/techstore
server {
    listen 80;
    server_name techstore.francecentral.cloudapp.azure.com;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl;
    server_name techstore.francecentral.cloudapp.azure.com;
    ssl_certificate /etc/letsencrypt/live/techstore.francecentral.cloudapp.azure.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/techstore.francecentral.cloudapp.azure.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    # Backend
    location /api/ {
        proxy_pass http://localhost:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
    # Frontend
    location /app {
        proxy_pass http://localhost:3002;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
    # MailPit
    location /mails/ {
        proxy_pass http://localhost:8025;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Deployed Application



Security Requirements and Tests Traceability

Security Requirement	Test
SR1 MFA	Postman Test: Try Login When User Has MFA; Postman Test: MFA Challenge
SR2 Lockout	Postman Test: SR2 Lockout (Attempt_1–Attempt_6)
SR3 Password policy	Postman Test: Register with Weak Password; Unit Tests: PasswordUtilsTest.java
SR4 Registration email	Manual: Test it on the prod environment
SR5 RBAC	Postman Test: Login As Customer; Postman Test: Try to access Manager Route
SR6 Session Expiry	Obs: Since the session timeout is configured in supabase, and to change it we needed pro account, and the default value is 1 hour, it was not possible to test it on a functional test, so we test it manually
SR7 Rate limiting	Postman Test: SR7 Rate limiting (Attempt_1–Attempt_6, Expect 429); Integration Test: RateLimitIntegrationTest.java
SR8 Encryption at rest and in transit	Postman Test: Encryption in Transit
SR9 Password hashing	Unit Tests: PasswordUtilsTest.java
SR10 GDPR compliance	-

Security Requirement	Test
SR11 Input validation	Postman Test: Register with Weak Password; Postman Test: Login With Wrong Data
SR12 Data format validation	Postman Test: Login With Wrong Data
SR13 Secure logs	Postman Test: Login With Wrong Creds
SR14 Log backup	Manual: See the backups on the vm